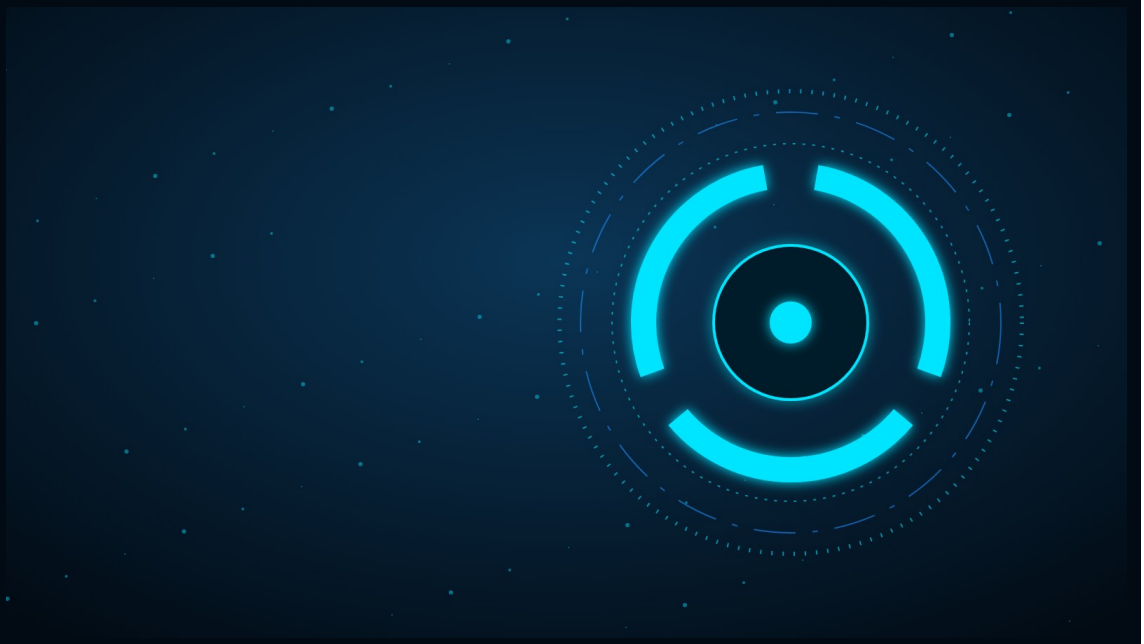
BATON

AI AND DATA SCIENCE | PROJECT REPORT

BATON

A multi-agent AI research platform that researches, analyses and writes reports



PREPARED BY

ANUSHAY AYAT

AI and Data Science, SMIT

November 2026

Acknowledgements

I would like to thank SMIT and my instructors for the guidance, knowledge and encouragement that made this project possible. I am also grateful to the open-source communities behind CrewAI, FastAPI and the other tools used here, and to my classmates for their feedback during development.

Anushay Ayat

Abstract

Baton is a web platform in which a crew of three specialised AI agents (a Researcher, an Analyst and an Editor) collaborates to turn a single topic into a structured research report. The user chooses the topic, report length and writing voice, then follows the work live on a futuristic mission-control dashboard. Finished reports are stored privately for each signed-in user.

Users sign in with email and password or with Google, and the site opens at the login page. The system exists in two modes. In local mode a FastAPI server runs a real CrewAI crew. In cloud mode the same three roles run as separate short API requests on Vercel, with Firebase Authentication and Firestore handling accounts and storage, and LangSmith tracing every agent run. This report presents the problem, requirements, design, implementation, user interface, deployment, testing and lessons learned.

Keywords: multi-agent systems, CrewAI, Groq, LangSmith, FastAPI, Firebase, Vercel, research automation

Contents

1 Introduction..... 4
2 Background and technology..... 5
3 Requirements analysis..... 6
4 System design..... 7
5 Implementation..... 10
6 User interface walkthrough..... 12
7 Deployment..... 18
8 Testing..... 19
9 Challenges and solutions..... 19
10 Security, privacy and ethics..... 20
11 Limitations and future work..... 20
12 Conclusion..... 20
Appendix A Local setup..... 21
Appendix B Environment variables..... 21
References..... 21

List of figures

Figure 1: Cloud architecture
Figure 2: Sequence of one mission
Figure 3: Data model and access rule
Figure 4: Landing page hero with navigation and key figures
Figure 5: Mission control ready: command bar, reactor and agent cards on standby
Figure 6: Mission control while an agent is working (amber)
Figure 7: All agents complete with time, words and output previews
Figure 8: The finished report with copy, download and print options
Figure 9: Features section
Figure 10: How it works: three steps
Figure 11: Frequently asked questions
Figure 12: Sign in page
Figure 13: Create account with password strength meter
Figure 14: Access granted confirmation after login
Figure 15: Responsive layout on a phone
Figure 16: Deployment pipeline

1 Introduction

1.1 Background

Researching a topic normally means searching, reading, analysing and writing, often across several tools. A single chatbot reply gives a quick answer but hides how it was produced and is hard to review, repeat or keep. Multi-agent systems split the work between specialists, which makes each step clearer and easier to inspect.

1.2 Problem statement

There is a need for a simple tool that guides a topic through research, analysis and writing, shows each stage as it happens, and keeps the results organised for each user.

1.3 Objectives

- Divide the work between three specialist agents with clear hand-offs.
- Show progress live with timings, word counts and previews.
- Provide secure accounts and a private report archive.
- Trace every agent run with LangSmith for debugging.
- Deploy the product on Vercel and Firebase with a professional interface.

1.4 Scope

The project covers the web interface, the agent pipeline, accounts, report storage, tracing and deployment configuration. It does not include payments, team sharing or automatic source verification, which are listed as future work.

1.5 Report organisation

Chapter 2 introduces the technologies. Chapter 3 states the requirements. Chapter 4 presents the design, and Chapter 5 the implementation. Chapter 6 walks through the interface. Chapters 7 to 9 cover deployment, testing and challenges. Chapters 10 to 12 discuss security, limitations, future work and the conclusion.

2 Background and technology

2.1 AI agents and multi-agent systems

An AI agent is a language model given a role, a goal and instructions. In a multi-agent system several agents work on parts of a task and pass results to each other. Giving each agent a narrow job usually makes behaviour easier to reason about than asking one prompt to do everything.

2.2 CrewAI

CrewAI is an open-source Python framework for organising role-playing agents into a crew that executes tasks in a defined process. Baton uses it in local mode with a sequential process, where each task receives the earlier tasks as context.

2.3 Groq and the GPT-OSS model

Groq provides hosted, fast inference for open models. Baton calls an open GPT-OSS model through Groq, configured by an environment variable so the model can be changed without editing code.

2.4 LangSmith

LangSmith records traces of language-model applications. Each run shows its inputs, outputs, timing and errors, which helps when debugging prompts or slow stages.

2.5 Firebase

Firebase Authentication provides email and password sign in. Cloud Firestore is a document database with security rules that can restrict access to the signed-in owner of a document.

2.6 Vercel

Vercel hosts static websites and serverless functions. Its documentation lists a 300 second maximum duration on the Hobby plan with Fluid compute and a 500 MB limit for Python functions, which influenced the cloud design described in Chapter 4.

3 Requirements analysis

3.1 Users

The main users are students, founders and analysts who want a structured starting point for a topic and want to see how it was produced.

3.2 Functional requirements

ID	Requirement
FR1	The user can enter a topic and choose report length (short, standard, deep) and voice (five options).
FR2	Three agents run in order: Researcher, Analyst, Editor, each using the earlier output.
FR3	The interface shows live status, timing, word count and a preview for every agent.
FR4	The finished report is shown as formatted text and can be copied, downloaded as Markdown or printed.
FR5	Users can create an account, sign in with email and password or with Google, and sign out.
FR6	Each user can reopen and delete their own past reports and cannot see anyone else's.
FR7	Every agent run is traced in LangSmith.
FR8	The site requires login: visitors who are not signed in are sent to the login page and see the main page after signing in.

3.3 Non-functional requirements

Quality	Requirement
Usability	Clear status at every step, helpful error messages, works on phones.
Security	Token-verified API calls, per-user data rules, secrets kept in environment variables.
Performance	Each agent runs as a short request that stays inside serverless time limits.
Maintainability	Model name, keys and origins are configuration; the code is split into small files.
Accessibility	Keyboard focus styles, text labels on controls, reduced-motion support.

4 System design

4.1 Architecture

The browser signs users in with Firebase Authentication and calls the Vercel API once per agent. The API verifies the Firebase ID token, runs the agent on Groq, and sends traces to LangSmith. The browser saves and loads reports in Firestore.

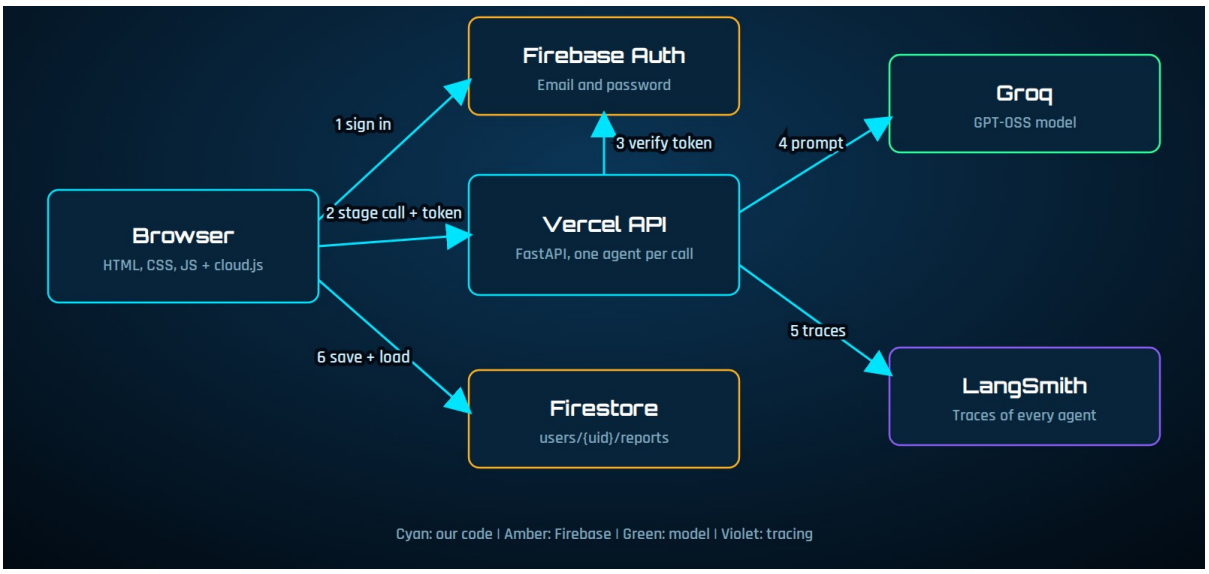


Figure 1: Cloud architecture

4.2 Agent relay

The output of each agent becomes the input of the next. The browser sends the earlier text with every call, so the API stays stateless.

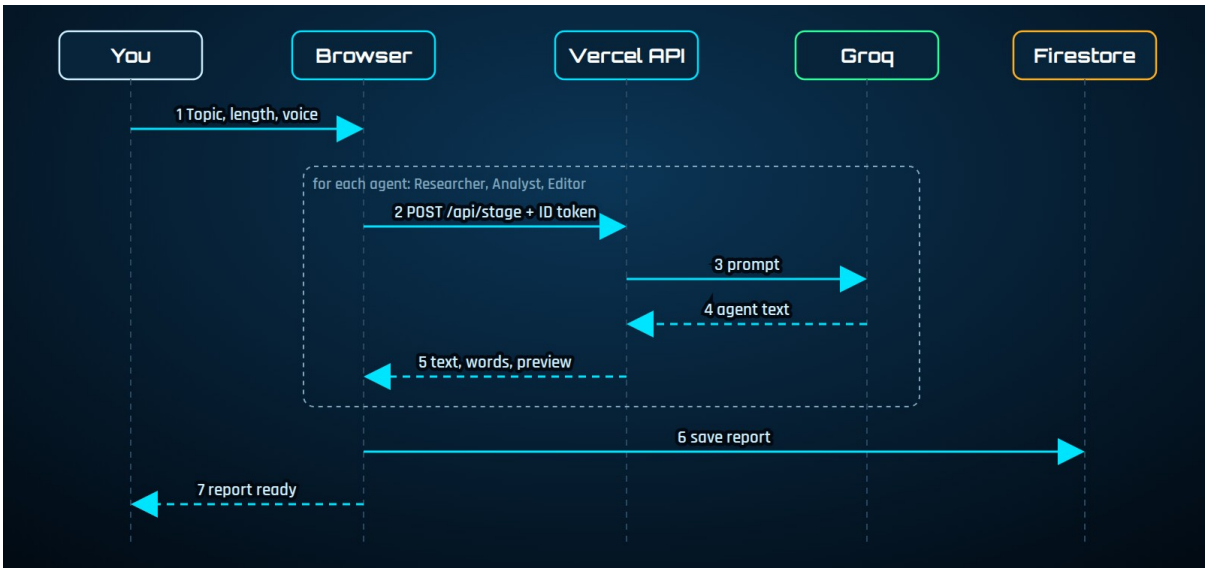


Figure 2: Sequence of one mission

Agent	Input	Responsibility
R-01 Researcher	Topic	Gathers facts, key concepts, developments, applications, benefits, challenges and future opportunities. Told not to invent facts or references.

Agent	Input	Responsibility
A-02 Analyst	Research notes	Produces insights, strengths, weaknesses, risks, opportunities and recommendations using only the research provided.
E-03 Editor	Notes and analysis	Writes the final Markdown report: executive summary, introduction, findings, analysis, benefits and challenges, recommendations, conclusion.

4.3 Data model

Reports are stored under the signed-in user. A Firestore rule allows reads and writes only when the request is authenticated and the user id matches the path.

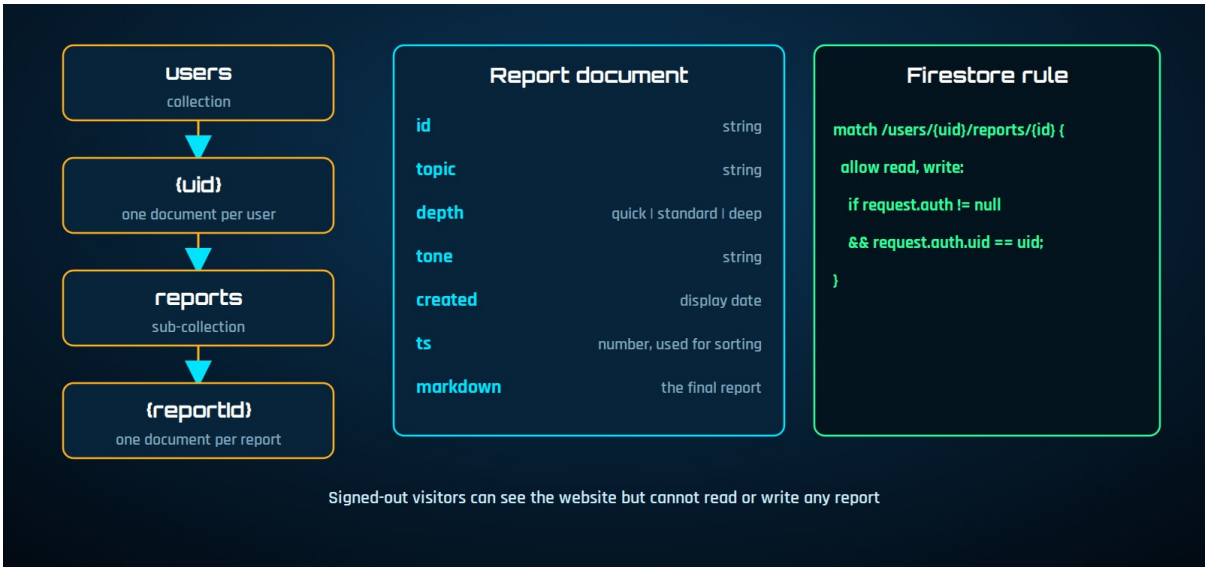


Figure 3: Data model and access rule

4.4 Local mode and cloud mode

Aspect	Local mode	Cloud mode
Agents	Real CrewAI crew	Same roles and prompts, one API call each
Server	main.py with FastAPI	api/index.py on Vercel
Accounts	SQLite with hashed passwords	Firebase Authentication
Reports	JSON files in reports/	Firestore under users/{uid}/reports
Tracing	One trace per crew run	One trace per agent

CrewAI is a heavy dependency and a full crew run can take minutes, so the cloud mode calls the model directly in three short requests. This keeps each request well inside serverless limits and gives natural live progress.

4.5 User interface design

The interface follows a heads-up-display style: glass panels with corner brackets, scanlines, a live particle network and an arc-reactor progress visual. Colour carries meaning: amber means an agent is working and cyan means it is done.

Colour	Value	Use
Void	#020B14	Page background
Panel	#07243A	Cards and panels
Cyan	#00E5FF	Primary accent, completed state
Blue	#1E90FF	Rotating rings
Amber	#FFB020	Agent working
Green	#2DFF9A	Online indicator

Headings use Orbitron and body text uses Rajdhani. Motion is disabled for users who prefer reduced motion.

5 Implementation

5.1 Project structure

```
crewai-baton/
  api/index.py            Vercel API (agent stages)
  public/index.html       Website and mission control (markup)
  public/login.html       Sign in and create account (markup)
  public/css/             hud, agents, effects, site, login
  public/js/              mission, agents, particles, effects, reveal,
                          nav-auth, auth-guard, login, cloud, firebase-config
  main.py                 Local server with CrewAI
  llm_patch.py            Groq cache_breakpoint fix
  vercel.json  firebase.json  firestore.rules
  requirements.txt        Cloud
  requirements-local.txt  Local
  DEPLOY.md  README.md
```

5.2 Frontend

The site is plain HTML, CSS and JavaScript, kept in separate files: HTML for markup, css/ for styles and js/ for behaviour. It has a sticky navigation bar, a hero section, mission control, features, a three-step explanation, use cases, a FAQ and a footer. Mission control shows a reactor with three arc segments (one per agent), three agent cards, a mission log and the report viewer. Reports are rendered from Markdown and sanitised before display.

5.3 Local backend with CrewAI

main.py defines the three agents and tasks and runs them with a sequential process. A small patch (llm_patch.py) removes a cache_breakpoint key that CrewAI adds to system messages, because Groq rejects it. Progress is published as events that the browser reads through a server-sent events stream.

5.4 Cloud API

A single endpoint, POST /api/stage, runs one agent. Each call verifies the Firebase ID token, runs the agent inside a LangSmith traced run and returns the text, word count and a short preview. The core of the implementation is shown below.

```
@traceable(run_type="llm", name="groq-completion", client=ls)
def llm(system: str, prompt: str) -> str:
    r = Groq(api_key=os.environ["GROQ_API_KEY"]).chat.completions.create(
        model=MODEL, temperature=0.3,
        messages=[{"role": "system", "content": system}, {"role": "user", "content":
prompt}])
    return r.choices[0].message.content or ""


@traceable(run_type="chain", client=ls)
def run_agent(role: str, system: str, prompt: str, topic: str = "") -> str:
    return llm(system, prompt)
```

Endpoint	Method	Description
/api/stage	POST	Runs the Researcher (0), Analyst (1) or Editor (2) after verifying the token
/api/health	GET	Reports model name, whether auth is on and whether tracing is enabled

5.5 Browser adapter

cloud.js connects the same interface to the cloud services. It calls the API three times, passing earlier output forward, and then saves the report to Firestore:

```
async function go(r, user) {
    const E = (t, x) => r.events.push({ type: t, ...x });
    let research, analysis, text;
    try {
      for (let i = 0; i < 3; i++) {
        E("stage", { index: i, state: "working" });
        const tok = await user.getIdToken();
        const res = await rf(API + "/api/stage", { method: "POST", headers: { "Content-
Type": "application/json", Authorization: "Bearer " + tok },
          body: JSON.stringify({ stage: i, topic: r.topic, depth: r.depth, tone: r.tone,
research, analysis }) });
        const d = await res.json();
        if (!res.ok) throw new Error(d.detail || "Request failed");
        if (i == 0) research = d.text; if (i == 1) analysis = d.text; if (i == 2) text =
d.text;
        E("stage", { index: i, state: "done", words: d.words, preview: d.preview });
      }
      const { fs, db } = await fb();
      await fs.setDoc(fs.doc(db, "users", user.uid, "reports", r.id), { id: r.id, topic:
r.topic, depth: r.depth, tone: r.tone,
        created: new Date().toLocaleString([], { day: "2-digit", month: "short", year:
"numeric", hour: "2-digit", minute: "2-digit" }), ts: Date.now(), markdown: text });
      E("done", { report_id: r.id });
    } catch (e) { E("error", { message: e.message }); }
  }
```

5.6 Authentication

Users sign up and sign in with Firebase Authentication, using email and password or a Google popup. In local mode the browser sends the Firebase ID token to the server, which verifies it and starts a session; in cloud mode the Firebase session is used directly. A small guard script (auth-guard.js) sends visitors who are not signed in to the login page, and the login page sends signed-in users to the main page. The login page shows clear errors, a password strength meter and an access-granted confirmation. The browser attaches the Firebase ID token to every agent request, and the API rejects calls without a valid token when authentication is enabled.

5.7 Observability with LangSmith

Each agent call is wrapped in a traced chain run with the model call nested inside it. Metadata records the agent name and user id, and tags mark the run as a Baton run. Tracing is switched on with three environment variables. In local mode the whole CrewAI crew run is traced as one run.

6 User interface walkthrough

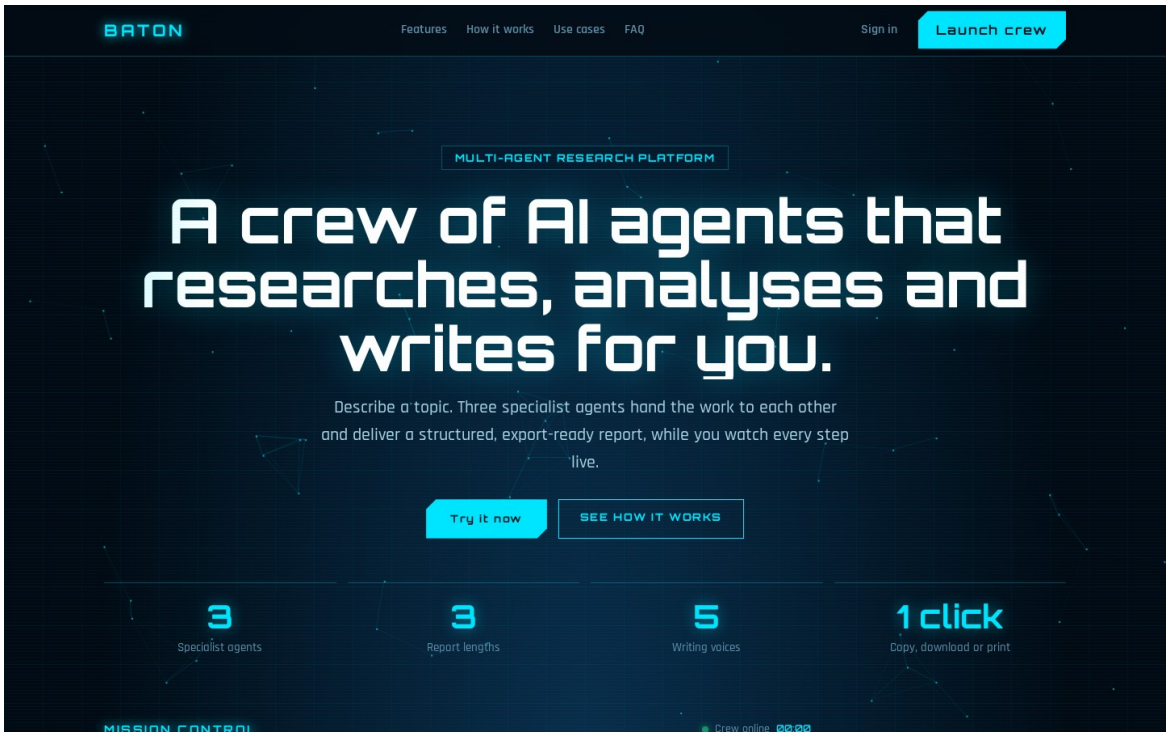


Figure 4: Landing page hero with navigation and key figures

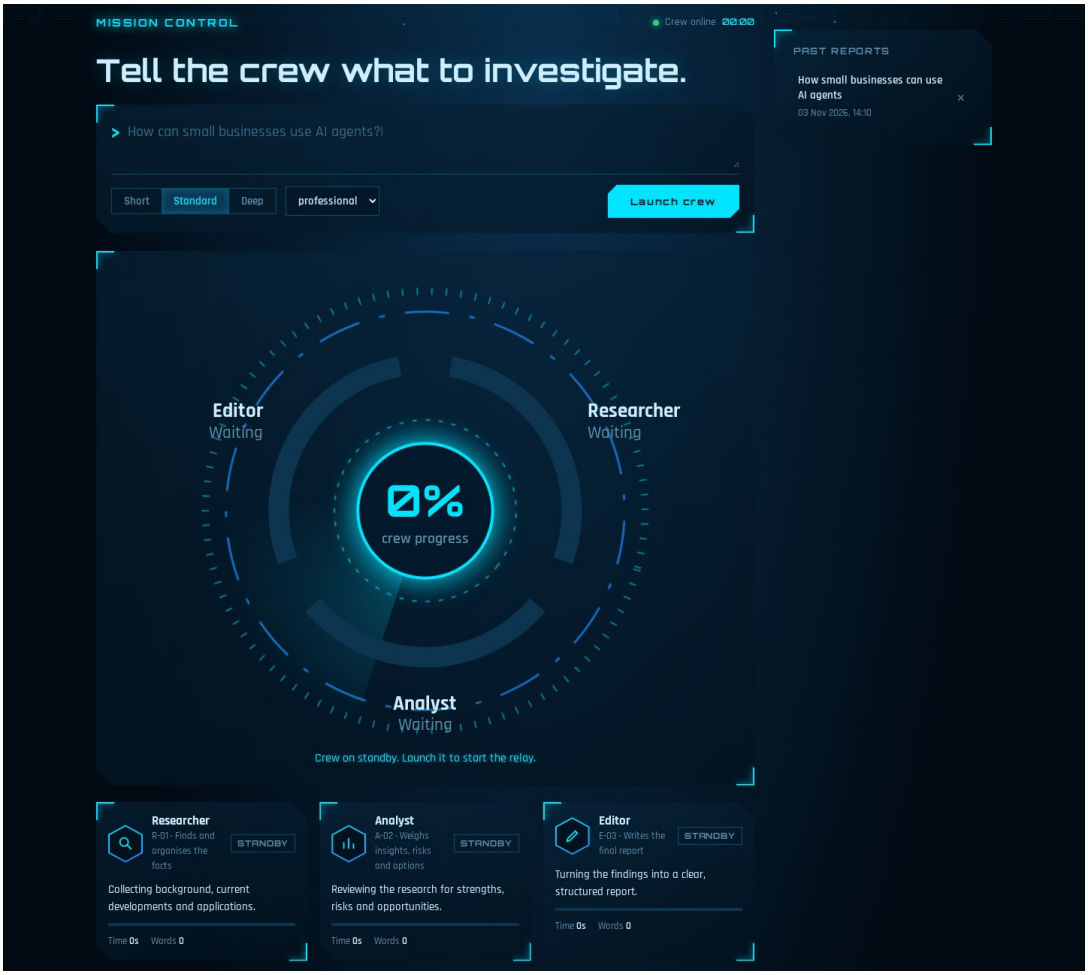


Figure 5: Mission control ready: command bar, reactor and agent cards on standby



Figure 6: Mission control while an agent is working (amber)

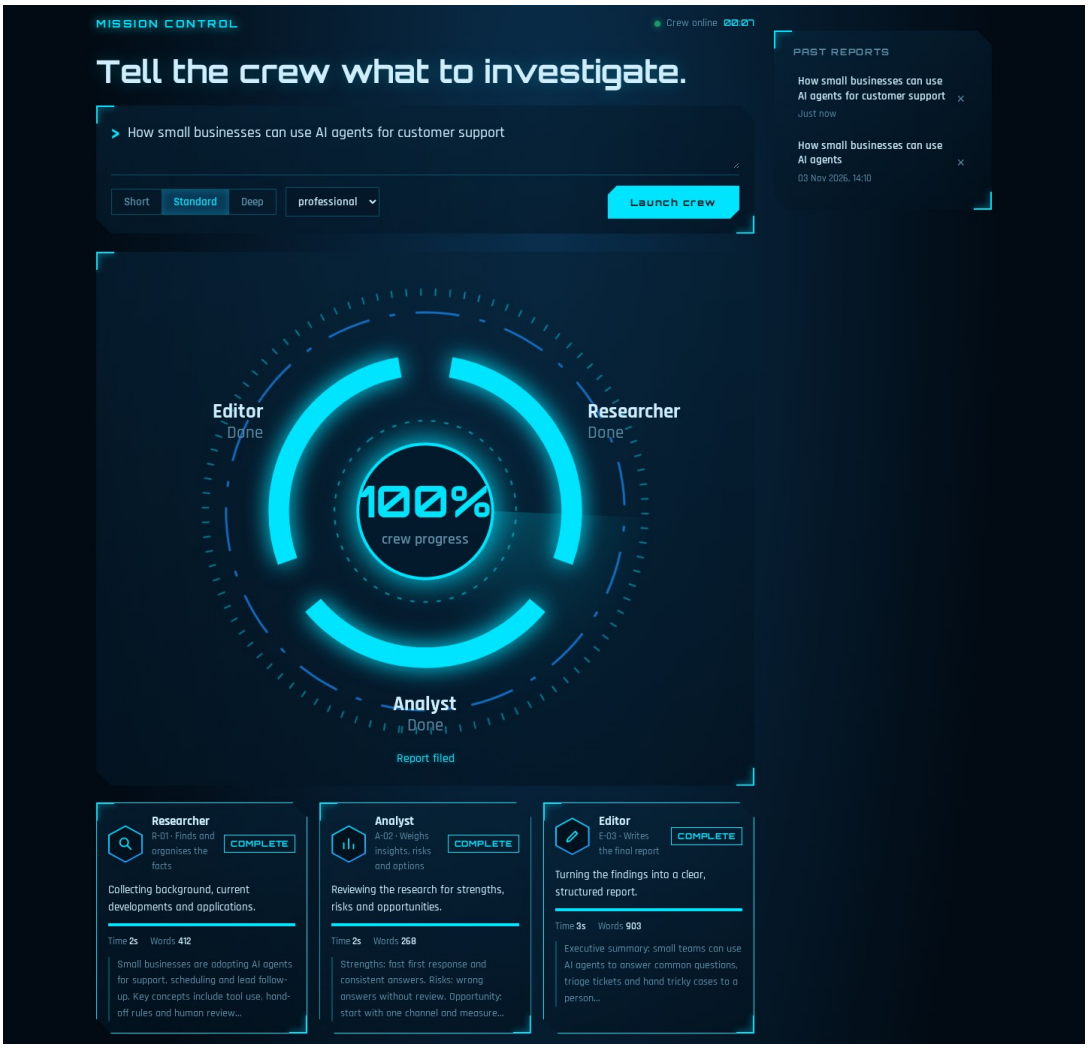


Figure 7: All agents complete with time, words and output previews

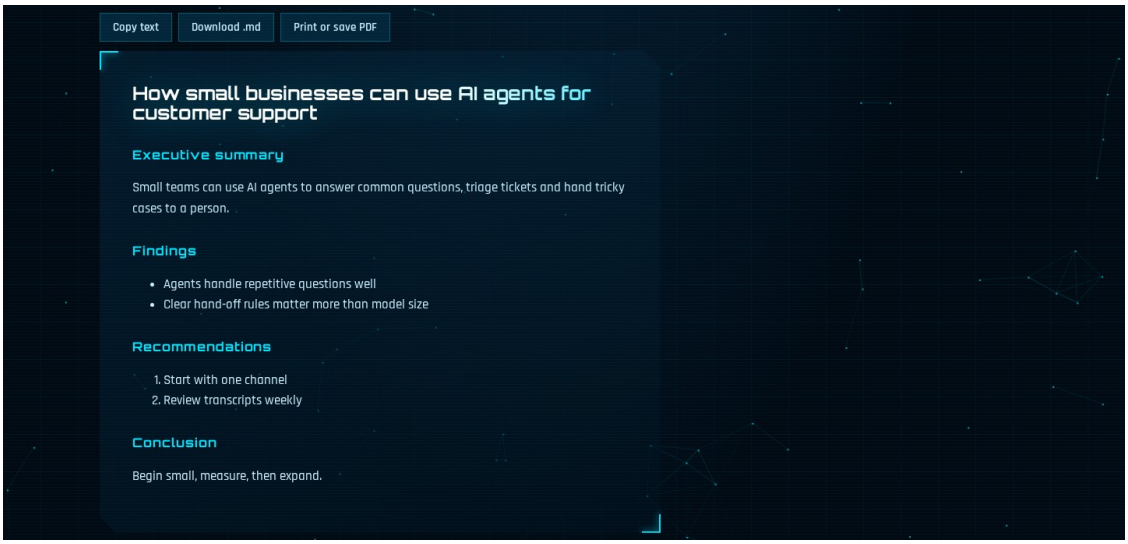


Figure 8: The finished report with copy, download and print options

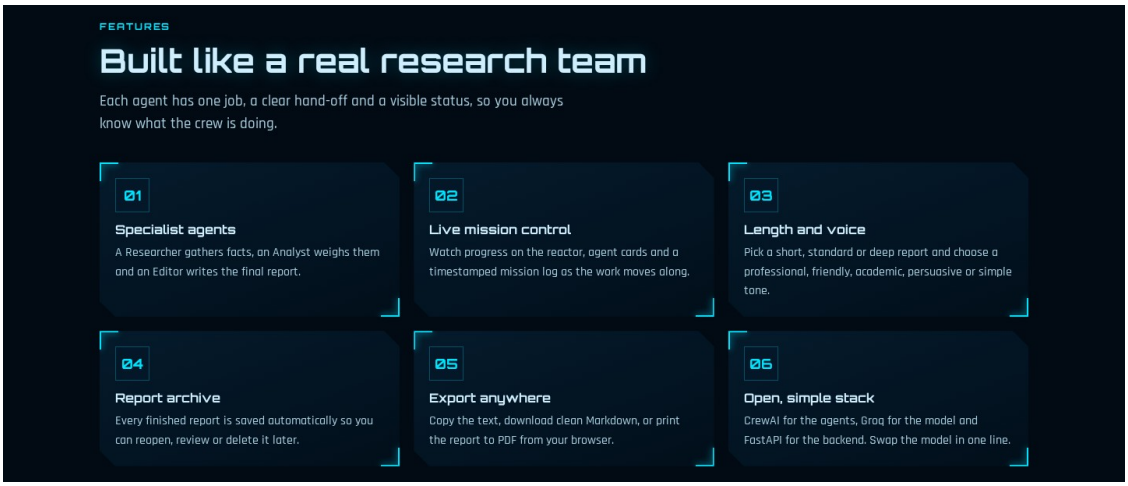


Figure 9: Features section



Figure 10: How it works: three steps



Figure 11: Frequently asked questions

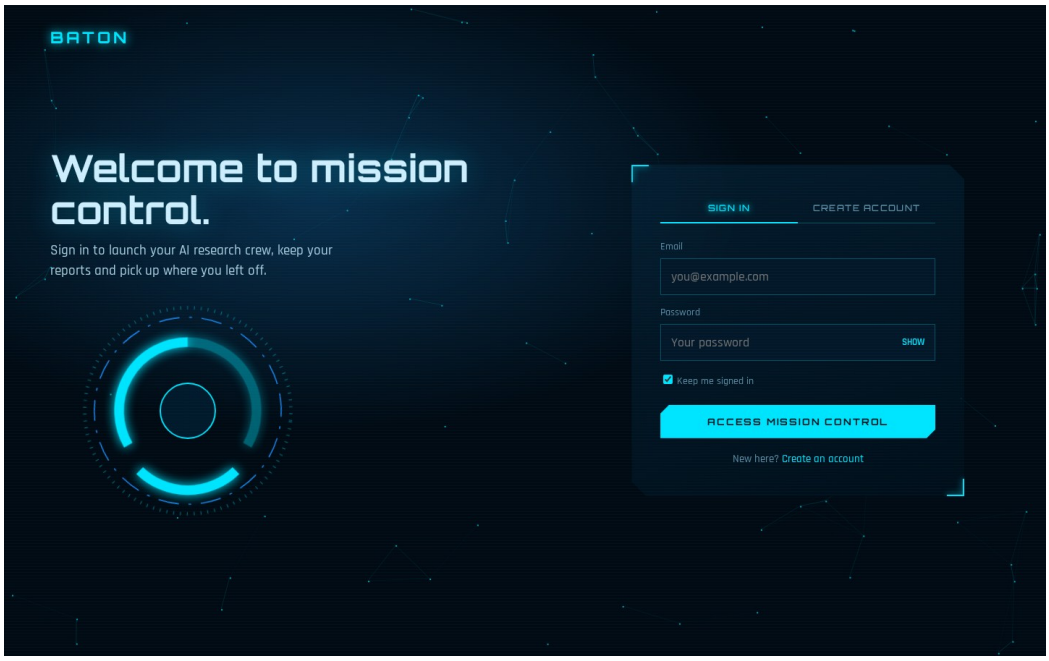


Figure 12: Sign in page

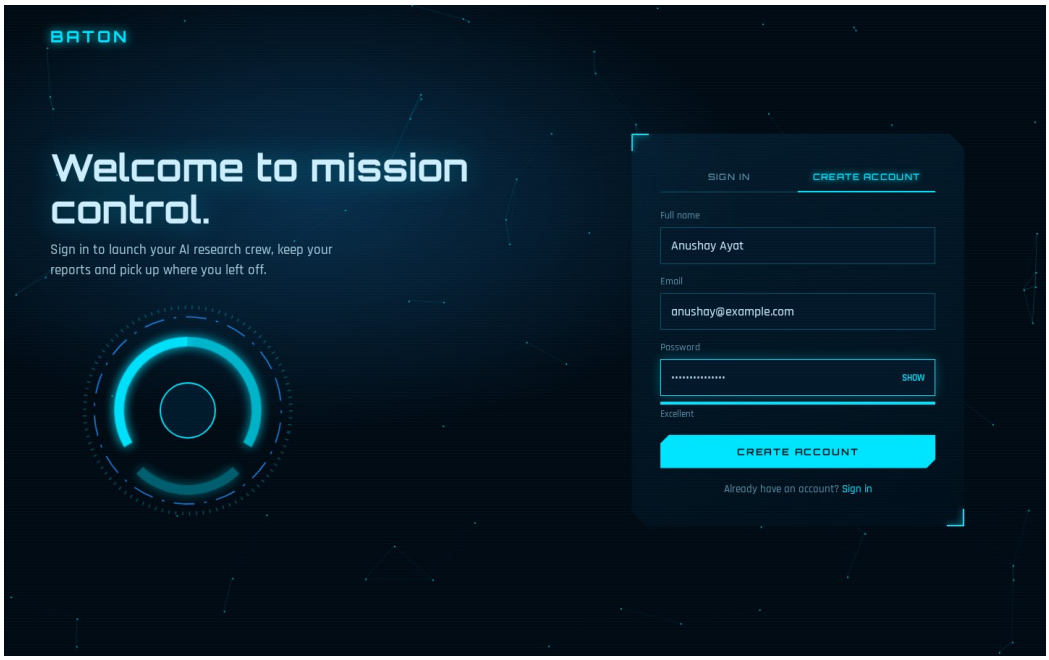


Figure 13: Create account with password strength meter

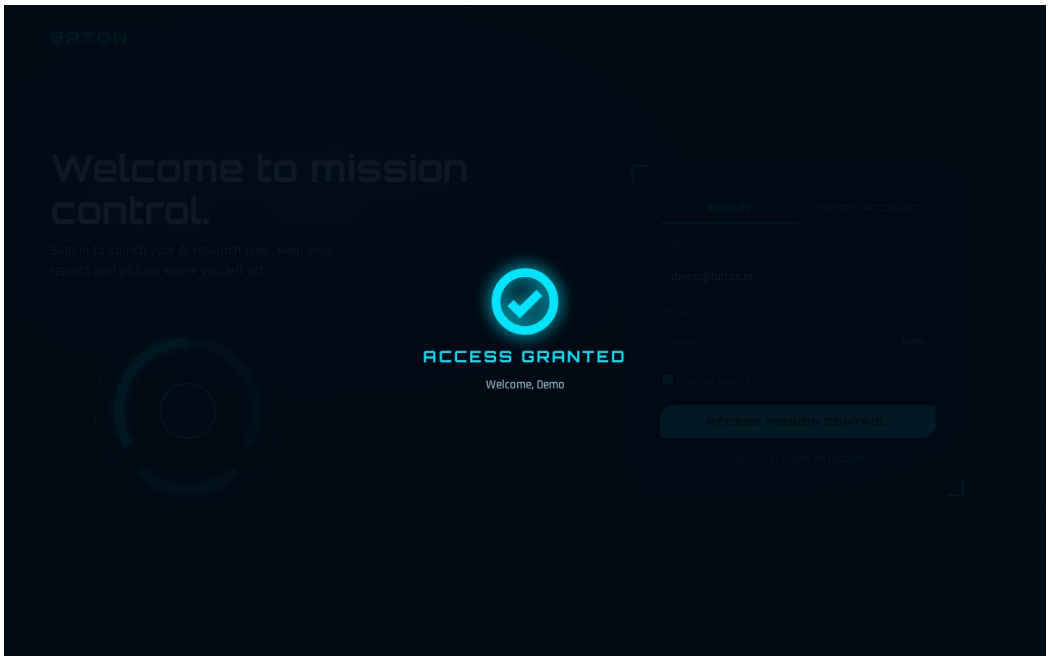


Figure 14: Access granted confirmation after login

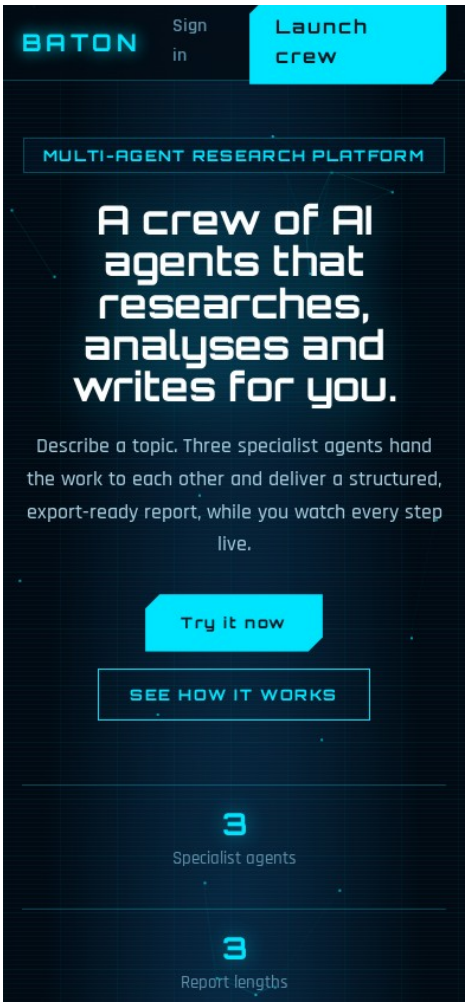


Figure 15: Responsive layout on a phone

The screenshots were captured from the demo build of the interface, which uses sample data in place of live model output.

7 Deployment

The project is deployed on Vercel (website and API) and Firebase (Authentication and Firestore). The live site runs at crewai-baton.vercel.app.



Figure 16: Deployment pipeline

- **Firebase:** create a project, enable Email/Password, create Firestore, paste the web config into firebase-config.js and publish firestore.rules.
- **LangSmith:** create an API key and a project named baton.
- **Vercel:** import the GitHub repository, add the environment variables and deploy.
- **Authorised domains:** add the Vercel domain in Firebase Authentication settings.
- **Verify:** open /api/health, create an account and launch a crew.

Problem	Fix
Redirected to login repeatedly	Add the domain to Firebase authorised domains
401 session expired	FIREBASE_PROJECT_ID on Vercel must match the Firebase project
502 from the API	Check GROQ_API_KEY and the model name in the Vercel logs
CORS error on Firebase Hosting	Set CORS_ORIGINS and BATON_API
No traces	Check the three LangSmith variables and redeploy

8 Testing

Automated and simulated tests were run during development, followed by a live run on the deployed site.

ID	Test	Method	Result
T1	API stage order	Stubbed model: stages 0, 1 and 2 return Researcher, Analyst and Editor	Passed
T2	API validation	Stage 1 without research returns 400	Passed
T3	API authentication	With auth on, a call without a token returns 401	Passed
T4	Cloud browser flow	Simulated browser with fake Firebase and API: login, launch, three stages, report saved	Passed
T5	History and sign out	Saved report appears in history; sign out and logged-out landing page behave correctly	Passed
T6	Login errors	Wrong password shows an error; correct password shows access granted	Passed
T7	Password strength	Strong password reports Excellent	Passed
T8	Script syntax	Syntax check of every inline and external script	Passed
T9	Live end-to-end	Deployed on Vercel with Firebase and Groq: Google sign-in and a full crew run	Passed
T10	Token claims	Firebase token check with realistic claims (user_id and sub); found and fixed a bug that read a missing uid field	Passed

All tests passed, including a live run on the deployed site at crewai-baton.vercel.app. Agent runs can also be reviewed as traces in the LangSmith project dashboard.

9 Challenges and solutions

Challenge	Cause	Solution
Circular import error	The file was named app.py and imported from app	Import from the crewai package instead
Groq rejected requests	CrewAI added a cache_breakpoint key to system messages	llm_patch.py removes the key before the request is sent
Serverless limits	CrewAI is heavy and a run takes minutes	Run each agent as a short API stage
No saved files in the cloud	Serverless storage is temporary	Store reports in Firestore per user
Buttons did not respond	A quoting error in the script stopped it from running	Script syntax checks and simulated browser tests
Session expired after login	The token check read a uid field that Firebase tokens do not contain	Read user_id or sub from the verified token

10 Security, privacy and ethics

- API keys are kept in environment variables and are never stored in the repository.
- The API verifies the Firebase ID token and Firestore rules isolate each user's reports.
- The Firebase web config is a public identifier; security relies on Authentication and the rules.
- Passwords in local mode are stored as salted hashes, never in plain text.
- Topics are sent to the model provider to produce a report, so users should avoid entering confidential information.
- AI output can contain mistakes. Reports are a starting point and important claims must be checked against trusted sources.

11 Limitations and future work

11.1 Limitations

- Cloud mode uses direct model calls instead of the CrewAI library, so behaviour can differ slightly from local mode.
- Reports do not include verified source links.

11.2 Future work

- Add more agents, such as a fact checker and a reviewer.
- Include source links and citations in reports.
- Add PDF export and report sharing.
- Add password reset and email verification.
- Evaluate report quality using LangSmith datasets.

12 Conclusion

Baton shows how a small crew of specialised agents can turn one question into a structured report while keeping the process visible and traceable. The project combines a multi-agent backend, secure accounts, per-user storage, tracing and a polished interface, and it is deployed on Vercel and Firebase.

Appendix A Local setup

```
pip install -r requirements-local.txt
copy .env.example to .env and add GROQ_API_KEY
python main.py
open http://127.0.0.1:8000
```

Appendix B Environment variables

Variable	Purpose
GROQ_API_KEY	Access to the Groq model
GROQ_MODEL	Model name, for example openai/gpt-oss-20b
FIREBASE_PROJECT_ID	Used to verify login tokens
LANGSMITH_TRACING	Turns tracing on
LANGSMITH_API_KEY	LangSmith access
LANGSMITH_PROJECT	Project name, for example baton
CORS_ORIGINS	Optional, used when the site is on Firebase Hosting

References

- CrewAI documentation: docs.crewai.com
- Groq documentation: console.groq.com/docs
- LangSmith documentation: docs.smith.langchain.com
- Firebase documentation: firebase.google.com/docs
- Vercel function limits: vercel.com/docs/functions/limitations